Structured Programming Lab

Bachelor Program Business Informatics

Department of Computer Science

Iteration Statements

Iteration Introduction

Iteration Motivation I

A way to deal with invalid input

```
1      Scanner sc = new Scanner(System.in);
2      int grade = sc.nextInt();
3      if (grade < 1 || grade > 5) {
4          System.out.printf("invalid.\n");
5          grade = sc.nextInt();
6      }
7      // classify grades
8      System.out.printf("Thank you.");
9      sc.close();
```

Iteration Motivation: Valid Input

```
2
Thank you.
```

Iteration Motivation: Invalid Input

```
9
invalid.
2
Thank you.
```

Iteration Motivation: FlowChart

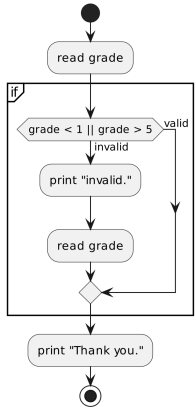


Figure 1: Flowchart of input validation with `if`

Iteration Motivation: Summary

- Only one invalid input can be handled
- We want to be able to **repeat** input (and validation) **until** input is **valid**
- We want to be able to **repeat** input (and validation) **while** input is **invalid**

`while` statement

WhileDemo - Code

```
1      Scanner sc = new Scanner(System.in);
2      int grade = sc.nextInt();
3      while (grade < 1 || grade > 5) {
4          System.out.printf("invalid.\n");
5          grade = sc.nextInt();
6      }
7      // classify grades
8      System.out.printf("Thank you.");
9      sc.close();
```

WhileDemo - valid input

```
2
Thank you.
```

WhileDemo - one invalid input

```
9
invalid.
2
Thank you.
```

WhileDemo - several invalid inputs

```
9
invalid.
12
invalid.
19
invalid.
102
invalid.
2
Thank you.
```

WhileDemo - flowchart

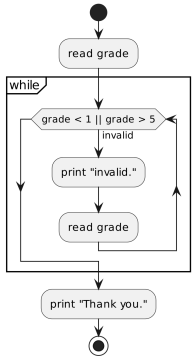
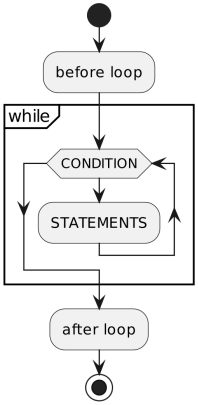


Figure 2: Flowchart of input loop. Note the difference to validation with `if`.

- Input is repeated **as long as** it is invalid
- Input is repeated **while** it is invalid

while statement - summary I




```
//before loop

while (CONDITION) {
    STATEMENTS
}

//after loop
```

Figure 3: Flowchart of the `while` loop

`while` statement - summary II

- Semantic:
  - `CONDITION` is always evaluated
  - if `CONDITION` is `true` the `STATEMENTS` are executed
  - and `CONDITION` is evaluated again
  - this cycle repeats *as long as* `CONDITION` evaluates to `true`
- `CONDITION` must be a `boolean` expression
- `CONDITION` is a **run condition**
- immediately after the loop `CONDITION` is `false` (otherwise we would still be *inside* the loop)

`while` statement - summary III

- `while` statement repeats a block of code
- `CONDITION` is evaluated at least once
- `STATEMENTS` are executed zero or more times
- **head-controlled** since `CONDITION` is evaluated **before** the **loop body** is executed

`for` statement

for statement - Motivation

A counter controlled while loop.

```
1     int n = 10;
2     while (n > 0) {
3         //many lines of complex code
4         System.out.printf("%d\n", n);
5         n--;
6     }
```

The loop

- starts at 10 (`int n = 10;`)
- ends at 1 (`n > 0`)
- in steps of 1 (`n--;`)

This information is not (or need not be) in the same line or in lines close to each other.

ForDemo - Code

```
1    for (int n = 10; n > 0; n--) {
2        //many lines of complex code
3        System.out.printf("%d\n", n);
4    }
```

The loop

- starts at 10 (`int n = 10;`)
- ends at 1 (`n > 0`)
- in steps of 1 (`n--;`)

This information is on the same line.

ForDemo - Console

```
10
9
8
7
6
5
4
3
2
1
```

ForDemo - Flowchart

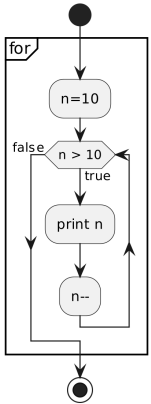
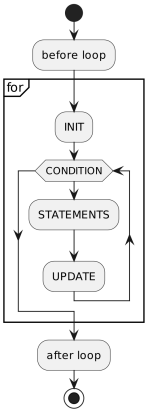


Figure 4: For Example

for statement - summary I




```
//before loop

for (INIT; CONDITION; UPDATE) {
    STATEMENTS
}

//after loop
```

Figure 5: Flowchart of the `for` loop

`for` statement - Summary II

- Semantic:
  - `INIT` is evaluated once
  - then `CONDITION` is evaluated
  - if `CONDITION` is `true` the `STATEMENTS` are executed and `UPDATE` is evaluated
  - then `CONDITION` is evaluated again
  - this cycle (`CONDITION -> STATEMENTS -> UPDATE -> CONDITION -> ...`) repeats *as long as* `CONDITION` evaluates to `true`
- `CONDITION` is a **run condition**
- `CONDITION` must be a `boolean` expression
- variables declared in `INIT` are local to the loop (out of scope after the loop)

`for` - Summary III

- `for` statement repeats a block of code
- it is typically used as a counter controlled loop
- `INIT` is evaluated exactly once
- `CONDITION` is evaluated at least once
- `STATEMENTS` are evaluated zero or more times
- `UPDATE` is evaluated zero or more times
- **head-controlled** since `CONDITION` is evaluated **before** the **loop body** is executed

do statement

DoDemo - Code

```
1      Scanner sc = new Scanner(System.in);
2      int grade;
3      do {
4          System.out.print(": ");
5          grade = sc.nextInt();
6      } while (grade < 1 || grade > 5);
7      // classify grades
8      System.out.printf("Thank you.");
9      sc.close();
```

DoDemo - valid input

```
: 2
Thank you.
```

DoDemo - one invalid input

```
: 9
: 2
Thank you.
```

DoDemo - several invalid inputs

```
: 9
: 12
: 19
: 102
: 2
Thank you.
```

DoDemo - flowchart

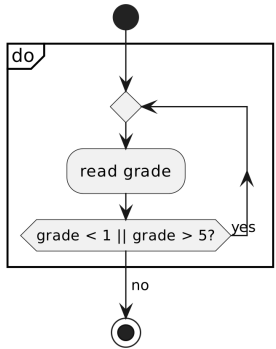
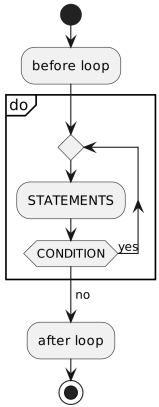


Figure 6: Flowchart of input loop with `do/while`.

- Input is repeated **as long as** it is invalid
- Input is repeated **while** it is invalid

do statement - summary I




```
1  //before loop
2
3  do {
4      STATEMENTS
5  } while (CONDITION); //note the ;
6
7  //after loop
```

Figure 7: Flowchart of the do loop

do statement - summary II

- Semantic:
  - `STATEMENTS` is executed at least once
  - `CONDITION` is evaluated at least once
  - if `CONDITION` is `true` the `STATEMENTS` are executed again
  - and then `CONDITION` is evaluated again
  - this cycle repeats *as long as* `CONDITION` evaluates to `true`
- `CONDITION` must be a `boolean` expression
- `CONDITION` is a **run condition**
- immediately after the loop `CONDITION` is `false` (otherwise we would still be *inside* the loop)

do statement - summary III

- do statement repeats a block of code
- `CONDITION` is evaluated at least once
- `STATEMENTS` are executed one or more times
- **foot-controlled** since `CONDITION` is evaluated **after** the **loop body** is executed

Special Cases

Infinite loops

```
while(true)
    ;
```

- the loop will never end (in a practical sense)
- similar constructs are in principle possible with the other iteration statements
- the above example is very simple - watch out for and avoid disguised infinite loops, like below

```
int a = 0;
while(a<10){
  //do smth
  a = 0; //typically some complicated calculation that always yields 0
}
```

Incomplete `for` I

```
int i=0;
for(; i<10; i++)
    ;
```

- `INIT` can be left blank

Incomplete `for` II

```
int i=0;
for(; i<10; )
    i++;
```

- `UPDATE` can also be left blank

Incomplete `for` III

```
for(;;)
    ;
```

- `CONDITION` can also be left blank
- it is the same as `for(; true; )` or `while(true)`
- this is an infinite loop

Jump commands

Break and Continue

- **`break`;**
  - can be used to jump out of a loop
  - is also used in `switch` to prevent "fallthrough" (see [Selection Statements])
- **`continue`;**
  - Control remains inside the loop, transfers control just before the end of the loop body
  - can be seen as "skipping" the current run and "continuing" with the next run

```
for (int i = 0; i < 10; i++) {
    if (i%2 == 1) continue; //skip odd numbers
    System.out.printf("%d\n", i);
}
```

Infinite loop with break

```
while (true) {
    statements;
    if(TERMINATION_CONDITION) break;
}
```

- `while(true)` would be an infinite loop, but the break command in the loop will end the loop
- control was transferred to the body of the loop
- it is possible to have several break commands at different positions in the loop body

Iteration summary

Iteration Overview I

- There are three iteration statements in Java
  - `while`
  - `for`
  - `do` (aka `do`/`while`)
- There is nothing one can do that the others cannot (all three have equal expressive power)
- It mght be easier to express something with one loop than with another
- for counter controlled loops (especially with [Arrays]) `for` loops are often preferred
- enclose the loop body in a block (`{ /* loop-body*/ }`)
- indent the body for clarity of structure and improved readability
- the loop body may consist of any sequence of statements, including
  - expression statements
  - input/output
  - selections (`if`/`else`)
  - iterations (loops)

Iteration Overview II

- jump statements let you tranfer control in non-standard ways
  - use `break` to “break out of a loop”
  - use `continue` to skip over the current run but stay in the loop
  - use them carefully and scarcely (only if you have a very good reason)